

Module V: Neural Reasoning, Model Compression and Federated Learning

What is Neural Reasoning ?

- Neural reasoning is **an AI approach where neural networks are designed or trained to execute logical, algorithmic, or multi-step reasoning tasks rather than simple pattern recognition.**
- It often involves embedding algorithms into neural architectures (Neural Algorithmic Reasoning) or combining neural networks with symbolic logic to improve generalizability, interpretability, and planning.

Key aspects of neural reasoning include:

- **Neural Algorithmic Reasoning (NAR):** Teaching neural networks to learn and execute classical algorithms (e.g., sorting, shortest path) on graphs or data, enabling them to generalize to unseen, larger inputs, often using **Graph Neural Networks** (GNNs).
- **Neurosymbolic AI:** Combining neural networks (for perception/data processing) with symbolic systems (for structured, logical reasoning) to create systems that can both recognize patterns and apply rules.

Neural Theorem Prover

Today’s large language models (LLMs) are exceptional at plausible language. They learn patterns, predict next words, and produce content that *sounds* right — useful for drafting, summarizing, or coding.

But they’re still **probabilistic pattern matchers**, not proof engines.
They can hallucinate.
They can’t give hard guarantees.

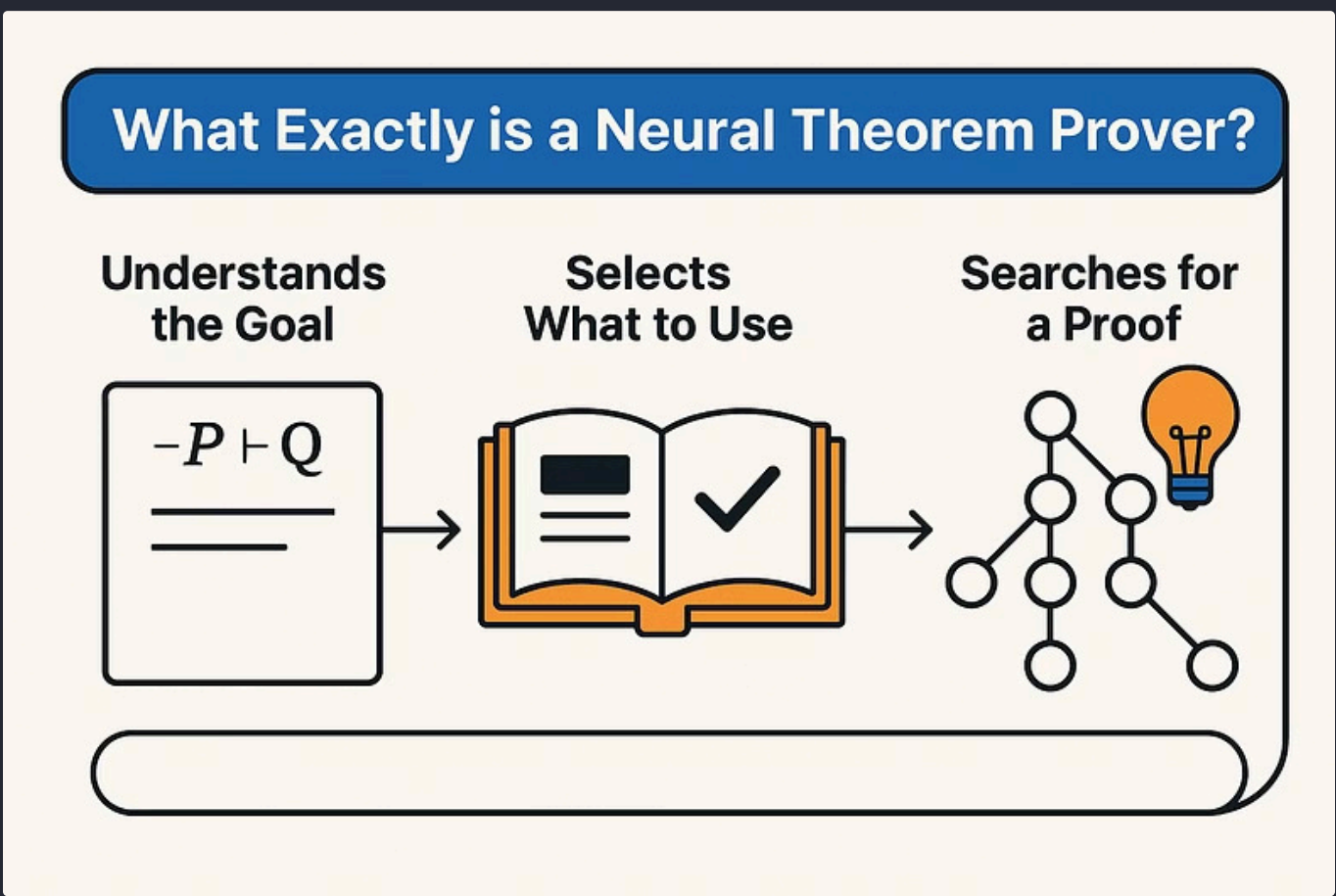
Formal mathematics lives in another universe:
every step must follow strict inference rules. A proof is either accepted or rejected — no “maybe.”

Traditional proof assistants like **Coq**, **Isabelle/HOL**, and **Lean** are reliable but notoriously hard to use — like ultra-strict lawyers: meticulous, trustworthy, and unforgiving.

Neural Theorem Provers bridge these worlds. They use neural networks to explore possible proofs while the proof assistant acts as a logical judge.

If the assistant accepts the steps, the result is no longer just a convincing story — it’s a **machine-checkable guarantee**.

What exactly is a Neural Theorem Prover?



A Neural Theorem Prover (NTP) does three core things:

Understands the goal

Receives a theorem — a structured logical statement the proof assistant understands.

Selects what to use (Premise Selection)

Chooses relevant lemmas from large formal libraries using neural semantic search.

Searches for a proof

- Explores a massive tree of proof states and tactics — rewriting, splitting, simplifying — guided by neural predictions, with every step verified by the proof assistant.
- Early systems used neural networks only for premise selection. Later ones added tactic choice and proof search.
- State-of-the-art models such as **DeepSeek-Prover-V2** integrate both **informal reasoning** (natural language) and **formal reasoning** (logical proofs) in a single loop — achieving **~89% accuracy**

How Neural Theorem Provers “think”?



Take the theorem: “*For any triangle, the sum of its interior angles is 180°.*”

A Neural Theorem Prover:

- 1

decomposes it into subgoals,
- 2

retrieves relevant lemmas (e.g., “angles on a straight line sum to 180°”),
- 3

applies tactics step by step,
- 4

learns from success and failure through feedback loops.

The distinction is profound:

- LLMs generate plausible text.
- NTPs generate verifiable sequences of logic.

That difference turns **explanations into evidence** — a foundation for **auditable logic** in enterprises.

Neuro-symbolic Reasoning frameworks

- Neuro-symbolic reasoning frameworks are hybrid systems that combine neural perception with symbolic logic to achieve transparent and modular reasoning.
- They employ modular pipelines that separate knowledge extraction, inference, and reasoning to facilitate task-specific adaptations and iterative self-refinement.
- Neuro-symbolic reasoning frameworks are **hybrid AI systems that combine the perceptual power of neural networks with the logical rigor of symbolic AI**. By integrating these two paradigms, these frameworks aim to achieve human-like cognition: the fast, intuitive pattern recognition of "System 1" (neural) and the slow, analytical reasoning of "System 2" (symbolic).

Core Components:

Core Components and Workflow

Most frameworks follow a modular pipeline that separates knowledge extraction from the actual reasoning process:

1. **Neural Perception Layer:** Processes raw input (images, text, sensor data) to extract high-level features or candidate entities.
2. **Symbol Grounding:** Maps neural outputs (e.g., a "0.98 dog" probability) to discrete symbolic representations (e.g., the symbol Dog).
3. **Symbolic Reasoning Layer:** Operates on these symbols using formal logic (e.g., First-Order Logic), knowledge graphs, or rule sets to derive conclusions or verify consistency.
4. **Feedback/Learning Loop:** Reasoning results are often fed back into the neural layer to regularize training or refine predictions based on logical constraints.

Classification of Integration Paradigms

Research taxonomies (notably by Henry Kautz and others) classify these frameworks based on how the neural and symbolic parts interact:

- **Loose Coupling (Symbolic-after-Neural):** Neural networks handle perception, and their outputs are passed to a symbolic engine for high-level decision-making (e.g., robotics navigation).
- **Tight Coupling (Neuro-Symbolic):** Symbolic rules are embedded directly into the neural architecture, often acting as a regularizer in the loss function to enforce logical consistency (e.g., **Logic Tensor Networks**).
- **Symmetric (Interleaved):** Both components engage in a bidirectional, iterative exchange, where symbolic reasoning helps the neural network learn more efficiently from less data.

Framework	Key Approach	Notable Features
DeepProbLog	Probabilistic Logic	Combines neural predicates with ProbLog for reasoning under uncertainty.
Scallop	Differentiable Datalog	A language designed for high-performance, differentiable relational reasoning.
Logic Tensor Networks (LTN)	Fuzzy Logic	Maps logical predicates to continuous vector spaces for neural processing.
DomiKnowS	Graph-based Logic	Models inference as an Integer Linear Programming (ILP) problem on a concept graph.
PyReason	Temporal Reasoning	Built for graph-based reasoning on top of neural network outputs.

Key Benefits

- **Explainability:** Provides a traceable "reasoning chain" rather than a black-box prediction.
- **Data Efficiency:** Symbolic priors (rules) allow systems to generalize from far fewer examples than pure deep learning.
- **Factual Reliability:** Explicit logic helps mitigate "hallucinations" in LLMs by grounding outputs in factual knowledge bases.

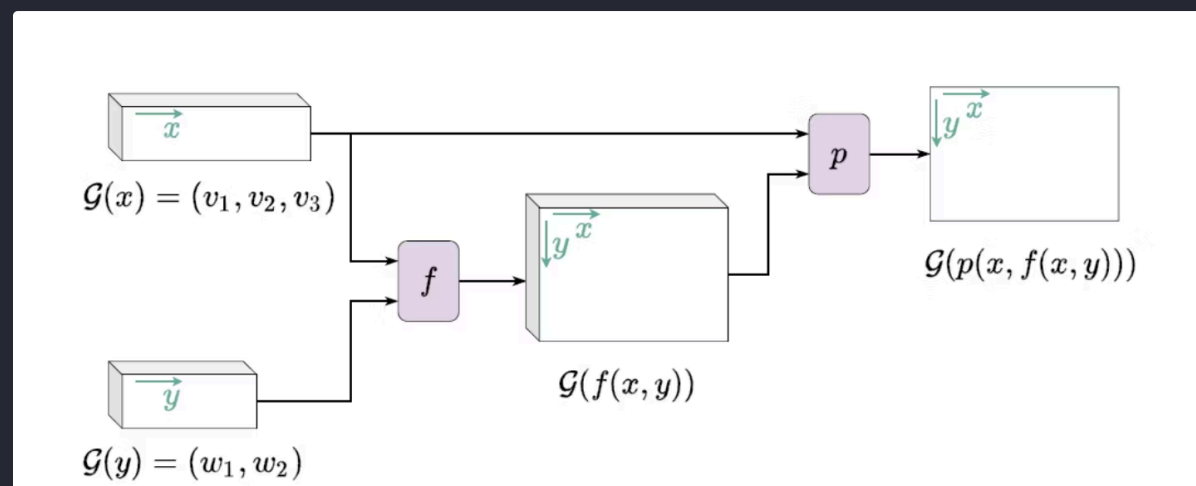
Logic Tensor Networks

Logic Tensor Networks (LTN) are a neurosymbolic artificial intelligence framework that merges the learning power of deep neural networks with the formal reasoning of first-order logic.

Core Concept: Real Logic

The foundation of LTN is a formalism called **Real Logic**, which maps logical symbols (constants, variables, and predicates) onto real-valued tensors and neural network operations. Unlike classical logic where statements are either true or false, Real Logic uses **fuzzy logic** semantics, where truth values range continuously between 0 and 1.

- **Grounding (G):** The process of mapping logical elements to concrete data. For instance, a "constant" might be grounded as a feature vector (embedding), and a "predicate" (like $IsCat(x)$) is grounded as a neural network that outputs a confidence score.
- **Differentiability:** Because all logical operations (AND, OR, NOT,) are implemented as differentiable functions (e.g., t-norms and aggregators), the entire system can be optimized using standard gradient descent.



How It Works

- **Define Knowledge:** You provide a set of logical axioms that the model should follow. for eg:

$$\forall x(\text{Dog}(x) \rightarrow \text{Mammal}(x))$$

- **Satisfiability as Loss:** The learning objective is to maximize the "satisfiability" of these axioms. The system calculates how much the current neural network weights violate the logic and uses this as a **logical loss function**.
- **Optimization:** The model updates its parameters so that the neural network's predictions become more consistent with both the provided data and the logical rules.

Differentiable Neural Computers

A **Differentiable Neural Computer (DNC)** is an advanced neural network architecture that combines the **pattern-recognition capabilities of deep learning** with the **structured data-storage power of a conventional computer**.

Introduced by [Google DeepMind](#) in 2016, a DNC is essentially a neural network "controller" augmented with a **dynamic external memory matrix**. Unlike standard neural networks that store knowledge only in their weights, a DNC can read from and write to this memory to solve complex reasoning tasks that require long-term storage, such as navigating a subway map or understanding family trees.

Core Architecture

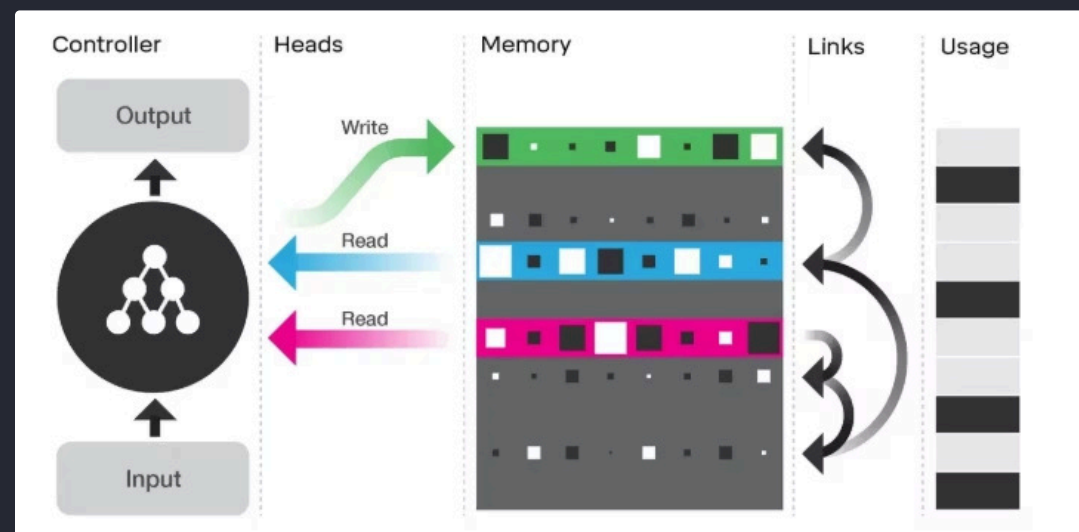
A DNC functions similarly to a Von Neumann architecture, where a processor (CPU) interacts with random-access memory (RAM).

- **Controller:** A neural network (typically an **LSTM** or feedforward network) that acts as the "brain," deciding what to read, write, or erase at each timestep.
- **External Memory:** A matrix ($N \times W$) that stores information in a way that is independent of the controller's internal state.
- **Differentiability:** Every operation—reading, writing, and memory allocation—is **fully differentiable**. This allows the entire system to be trained end-to-end using gradient descent.

Key Mechanisms

The DNC uses specialized "heads" to manage memory interaction:

- **Content-Based Addressing:** The controller searches memory for information that is similar to a specific "key".
- **Temporal Linkage:** The model tracks the order in which information was stored, allowing it to "remember" sequences of events forward or backward in time.
- **Dynamic Allocation:** The controller can automatically find unused memory locations to store new data or "free up" locations it no longer needs.



Memory-augmented Neural Networks

Memory-Augmented Neural Networks (MANNs) are a class of artificial neural networks that integrate an external memory component with a standard neural network architecture. Unlike traditional networks like **Recurrent Neural Networks (RNNs)** or **LSTMs** which encode information within internal weights or hidden states, MANNs can store and retrieve data from a separate **differentiable memory matrix**

The specific "differentiable points" that set the DNC apart from more basic MANNs (like the original Neural Turing Machine) are:

1. **Temporal Linking (Memory Order) Basic MANN:** Usually stores data as a collection of vectors. It can find what is in memory based on content, but it often forgets the order in which things happened. DNC: Uses a Temporal Linkage Matrix. This allows the model to remember the sequence of information. It can "look back" to see what was written before a specific memory or "look forward" to see what came next, which is vital for tasks like navigating a map or following instructions.
2. **Dynamic Memory Allocation Basic MANN:** Often writes to memory by overwriting what was there or using simple location-based shifts. It isn't very good at finding "empty" space. DNC: Includes a Usage Vector that tracks which memory slots are currently being used and which are free. This allows the DNC to dynamically allocate memory to new information without accidentally deleting important old data—much like how a real computer's operating system manages RAM.
3. **Advanced Attention Mechanisms** While all MANNs use differentiable attention to "read" and "write," the DNC uses a hybrid approach: Content-based addressing: Finding memory that looks like the current input. Temporal-based addressing: Finding memory based on when it was stored.

Pruning

Pruning is a deep learning model compression technique that removes unnecessary weights, channels, or neurons with low-impact on accuracy, resulting in smaller, faster, and more efficient models

What is Neural Network Pruning?

Neural network pruning is a process of removing redundant or less important neurons or connections (weights) from a neural network without significantly impacting its performance. By eliminating these unnecessary components, the model's size and complexity can be reduced, leading to faster inference times and lower memory usage.

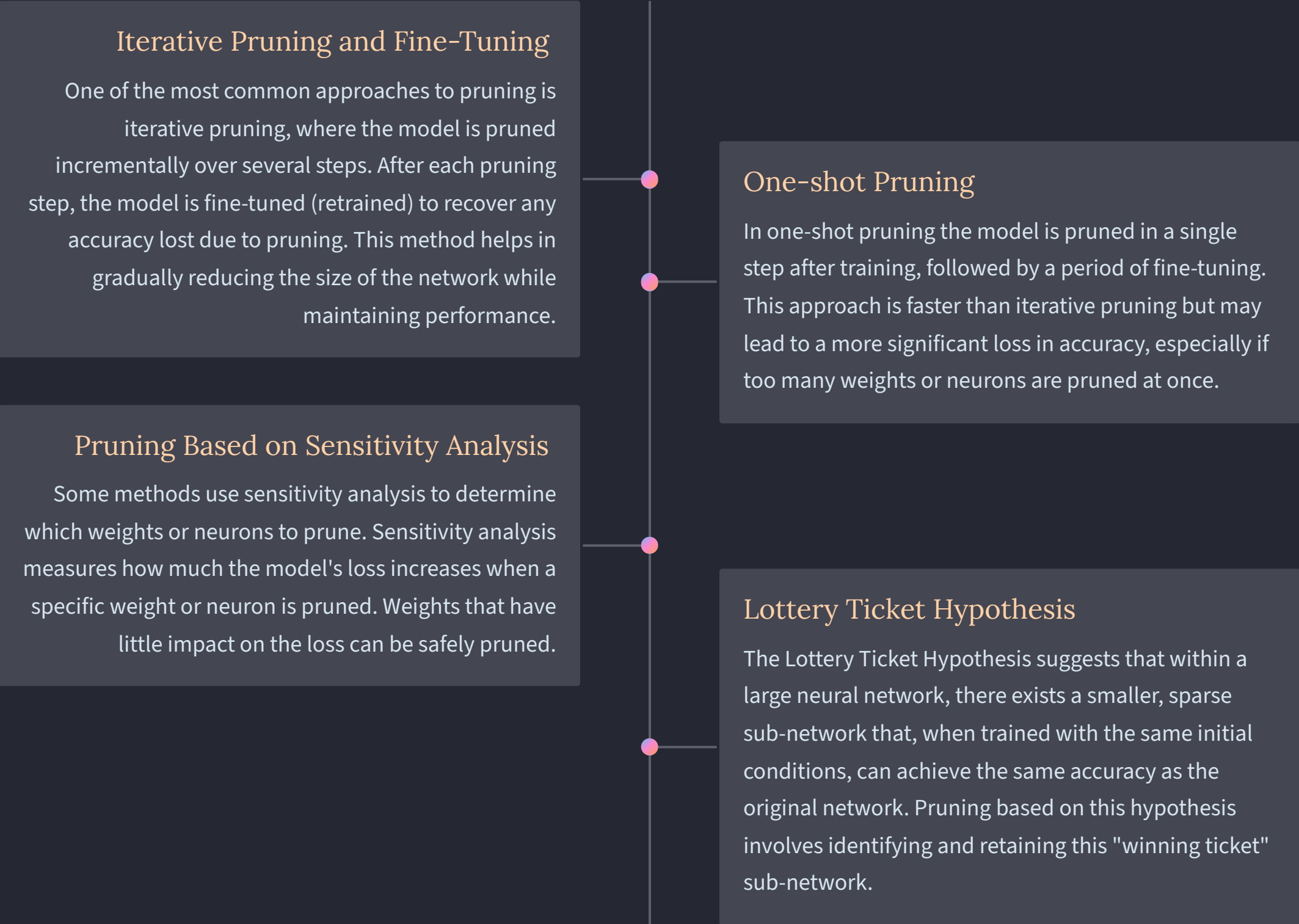
Types of Neural Network Pruning

There are several types of neural network pruning techniques, each targeting different components of a neural network. The most common types are weight pruning, structured pruning, and layer pruning.

01	02	03
Weight Pruning (Unstructured Pruning) Weight pruning is the most basic form of pruning, where individual connections (weights) between neurons are pruned. This method removes weights that have little or no contribution to the final predictions. By zeroing out these weights, the network becomes sparse, meaning that only a fraction of the connections are active during inference. - Magnitude-based Pruning In this method, weights with small magnitudes (close to zero) are considered less important and are pruned. The assumption is that these weights have minimal impact on the model's predictions. - Random Pruning: Here, weights are pruned randomly without considering their value. This technique is not commonly used due to the risk of removing important weights.	Structured Pruning In structured pruning, entire groups of weights, neurons, or filters are pruned instead of individual weights. This method is often more effective for reducing the computational cost because it leads to models that can be efficiently run on modern hardware like GPUs or TPUs. - Filter Pruning This method prunes entire convolutional filters in CNNs. Filters that are not contributing significantly to the output are removed, leading to smaller and faster models. - Neuron Pruning: Neurons that are contributing less to the output are pruned. This is particularly useful in fully connected layers, where many neurons may be redundant. - Layer Pruning: In some cases, entire layers or blocks of layers can be pruned, especially if they are contributing little to the model's accuracy. However, this technique is more aggressive and can result in a significant loss of performance if not done carefully	Dynamic Pruning In dynamic pruning, the pruning process is adaptive and occurs during training rather than after the model has been fully trained. This allows the model to learn which connections are important and which can be removed as training progresses.

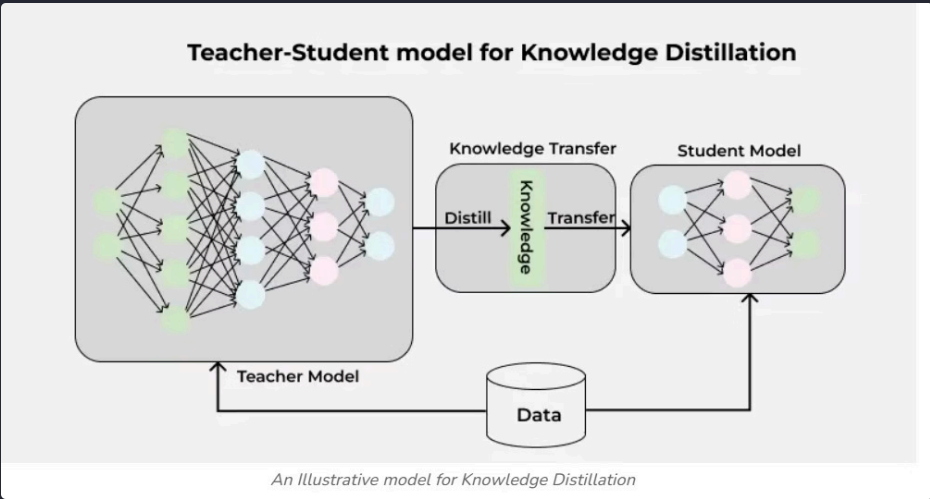
Methods for Neural Network Pruning

Various methods have been developed for pruning neural networks. These methods vary in complexity and the level of control they offer over the pruning process.



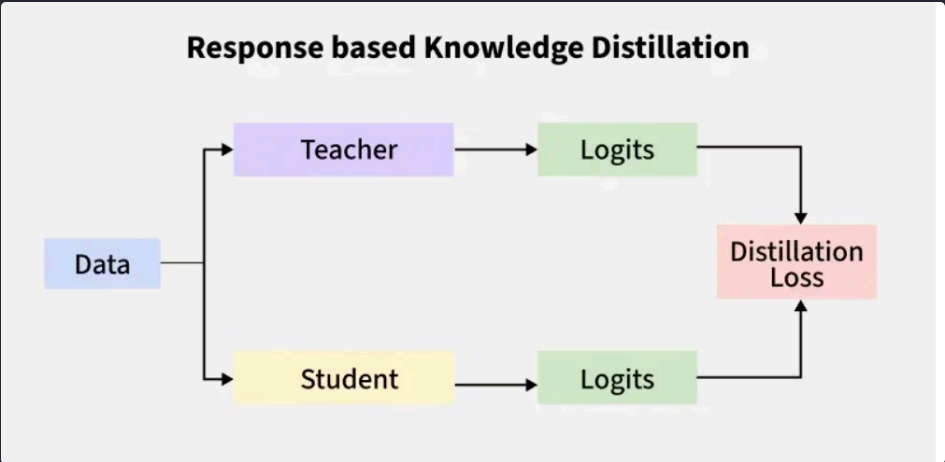
Knowledge Distillation

Knowledge Distillation is a model compression technique in which a smaller, simpler model (student) is trained to imitate the behavior of a larger, complex model (teacher). Instead of learning directly from data, the student model learns from the soft targets or probabilities, which are produced by the teacher model. This technique helps in deploying deep models on edge devices like mobiles or IOT.



Types of Knowledge Distillation

Response-based Distillation



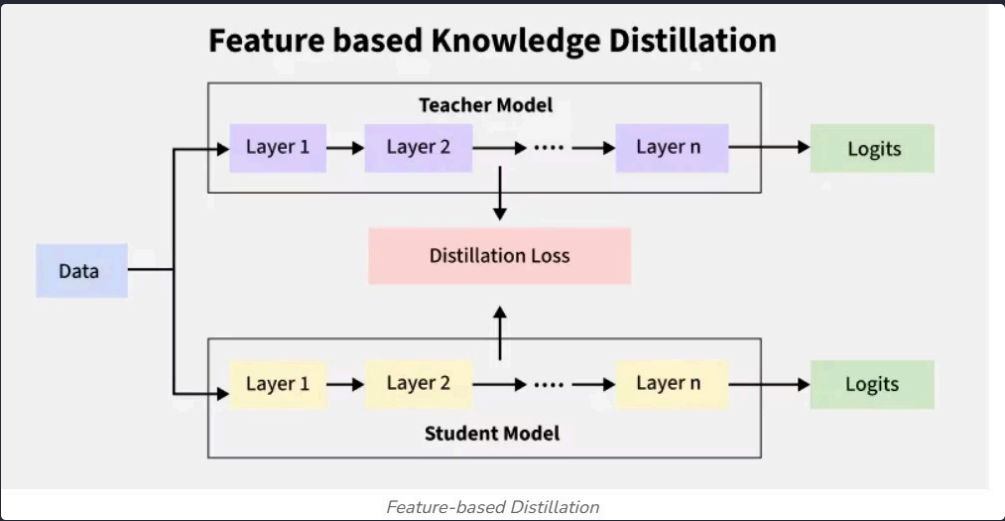
Response-based Distillation

This is the most classic and widely used form of knowledge distillation. It transfers the softened output probabilities (logits) from the teacher model to the student. These outputs provide richer information than hard labels because they reflect class similarities learned by the teacher.

- Relies on softmax outputs with a temperature parameter.
- Uses KL divergence loss to align student predictions with teacher.
- Does not require access to internal layers of the teacher.
- Simple and easy to implement.
- Highly effective in classification tasks.

Feature-based Distillation

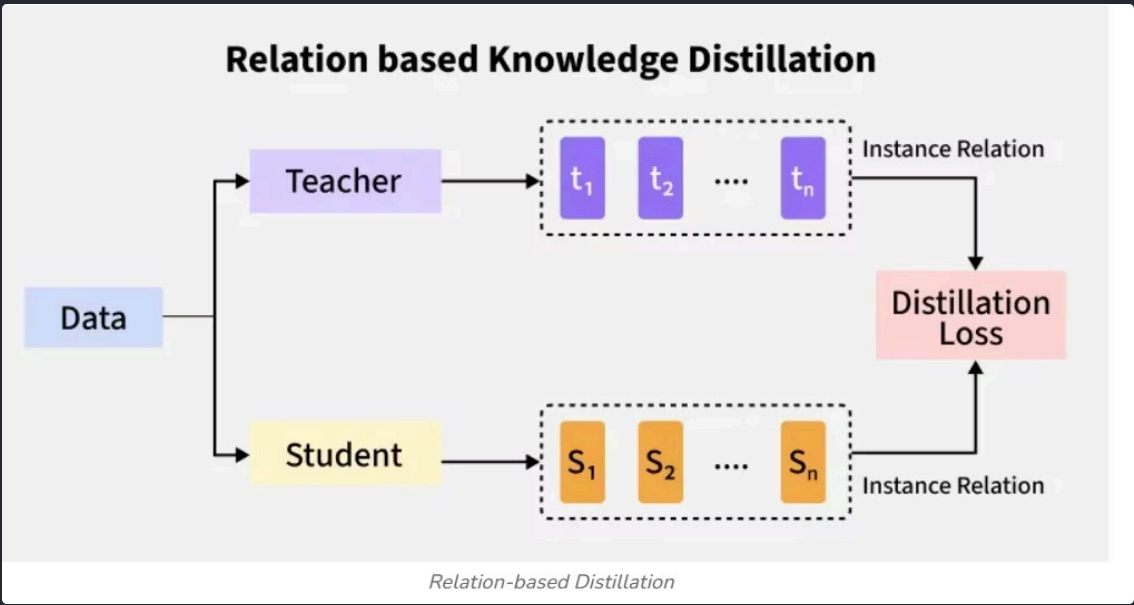
Feature-based-Knowledge-Distillation Feature-based Distillation Instead of using final outputs, this method transfers intermediate representations or feature maps from the teacher to the student. The idea is to guide the student to learn similar internal feature structures.



Matches hidden layer activations between teacher and student. Can use L2 loss or attention maps for alignment. Useful in vision and NLP models (e.g., CNNs, Transformers). Allows deeper guidance than response-based KD. Requires access to teacher’s internal layers.

Relation-based Distillation

Relation-based-Knowledge-Distillation Relation-based Distillation This technique focuses on the relationships between different input samples, such as pairwise distances or similarities. The goal is to maintain the relative structure of the learned space.



Transfers relationships between instances, like distances or similarities between samples. Captures teacher’s understanding of sample distribution. Helps in tasks like metric learning and representation learning. Independent of specific output predictions. Requires calculating inter-sample statistics.